Software Architecture & Design Specification (SADD / SDS)

Webhook Aggregator System (WACDP)

Document Identifier: `SADD-WACDP-2026-V1.0`

Standard: IEEE Std 1016-2009 (Software Design Descriptions)

Project Name: Webhook Aggregator Core Development Pipeline

Batch / Topic Code: BPS49 — Webhook Aggregator

Project Team / Authors:

- **Sumedh Girish** (SRN: `PES1UG24CS480`)
- **Subramani B M** (SRN: `PES1UG24CS473`)
- **Sujan S Halanannavar** (SRN: `PES1UG24CS477`)
- **Skanda Shyam Nadig** (SRN: `PES1UG24CS458`)

Institution: Department of Computer Science & Engineering, PES University

Project Code: BPS49 — Webhook Aggregator Core Development Pipeline

Date: September 2026

Status: Approved / Architecture Baseline Version 1.0

Table of Contents

1. Introduction
- 1.1 Purpose
- 1.2 Scope & System Overview
- 1.3 Definitions, Acronyms and Abbreviations
- 1.4 References

2. Architectural Overview & System Context
- 2.1 System Context Diagram
- 2.2 Design Constraints & Guiding Principles

3. Architecture: Component Model & Descriptions
- 3.1 Component Diagram
- 3.2 Detailed Component Specifications

4. Architecture: Architectural Patterns
- 4.1 Event-Driven Architecture (EDA) & Reactive Buffering
- 4.2 Multi-Queue Producer-Consumer Pattern
- 4.3 Worker Thread Pool & Shared Concurrent Task Queue Pattern
- 4.4 Exponential Backoff & Circuit Protection Pattern
- 4.5 Dead Letter Channel & Persistent Audit Trail Pattern

5. Architecture: Traceability to Requirements

6. Architecture: Security Architecture
- 6.1 Defense-in-Depth Multi-Layer Model
- 6.2 Egress Payload Signing (HMAC-SHA256)
- 6.3 Ingestion Perimeter Defense & Rate Throttling

• 6.4 Transport & Data-at-Rest Protection
7. Architecture: Other Architectural Sections
• 7.1 Deployment & Physical Topology View
• 7.2 Concurrency & Thread Synchronization Architecture
8. Design: UML Sequence Diagrams
• 8.1 Sequence Diagram 1: One-Shot Webhook Ingestion, Buffering, Worker Processing & Dispatch
• 8.2 Sequence Diagram 2: Scheduled Recurring Webhook, Target Outage, Exponential Backoff & DLQ Persistence
9. Design: Detailed Workflow & Activity Diagram
• 9.1 Multi-Swimlane Activity Diagram Analysis
10. Design: API Design Specification
• 10.1 Ingestion API (`POST /api/v1/webhooks/ingest`)
• 10.2 Recurring Schedule API (`POST /api/v1/webhooks/recurring`)
• 10.3 Status Query API (`GET /api/v1/webhooks/{task_id}/status`)
• 10.4 DLQ Inspection API (`GET /api/v1/webhooks/failed`)
• 10.5 Manual Redrive API (`POST /api/v1/webhooks/{task_id}/retry`)
• 10.6 System Health API (`GET /api/v1/health`)
11. Design: Error Handling & Fault Tolerance
• 11.1 Error Classification & Fault Taxonomy
• 11.2 Exponential Backoff Algorithm Formulation
• 11.3 Dead Letter Storage & Poison Pill Isolation
12. Design: Data Dictionary & Memory Schemas
• 12.1 In-Memory Buffer Task Schema
• 12.2 Scheduler Timer Handle Schema
• 12.3 Persistent DLQ Database Schema

1. Introduction

1.1 Purpose

This Software Architecture & Design Specification (SADD / SDS) provides a comprehensive, formal blueprint of the structural components, behavioral patterns, interaction sequences, interface protocols, and security models governing the **Webhook Aggregator System (WACDP)**. It translates the requirements established in `SRS-WACDP-2026-V1.0` into concrete architectural entities and design solutions, conforming strictly to **IEEE Std 1016-2009**.

1.2 Scope & System Overview

The Webhook Aggregator System is a high-performance middleware application engineered to ingest, buffer, schedule, compute, and reliably dispatch webhooks across enterprise microservices. It bridges asynchronous event submission with reliable asynchronous delivery with bounded retries and durable DLQ persistence, eliminating point-to-point network coupling, absorbing traffic spikes, and offering high-resilience dispatching through exponential backoff retry loops and durable dead-letter auditing.

1.3 Definitions, Acronyms and Abbreviations

- **WACDP:** Webhook Aggregator Core Development Pipeline.
- **DLQ:** Dead Letter Queue (durable storage for failed webhook payloads).
- **HMAC:** Hash-based Message Authentication Code.
- **EDA:** Event-Driven Architecture.
- **FIFO:** First-In, First-Out.
- **p99:** 99th percentile response time.

- **TLS:** Transport Layer Security.

1.4 References

1. IEEE Std 1016-2009: *IEEE Standard for Information Technology — Systems Design — Software Design Descriptions*.
2. `SRS-WACDP-2026-V1.0`: *Software Requirements Specification for Webhook Aggregator System*.
3. `STP-WACDP-2026-V1.0`: *Software Test Plan & Test Cases for Webhook Aggregator System*.
4. *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions* (Gregor Hohpe & Bobby Woolf).

---

2. Architectural Overview & System Context

2.1 System Context Diagram

The Webhook Aggregator System sits at the boundary between upstream event-generating applications and external webhook destinations:



2.2 Design Constraints & Guiding Principles

1. **Low-Latency Non-Blocking Ingestion:** The ingress event loop must never block waiting for disk I/O, downstream network calls, or worker threads.
2. **Decoupled Asynchrony:** Upstream clients receive immediate HTTP 202 Accepted status codes; all downstream delivery and retries occur asynchronously.
3. **Thread Safety Without Starvation:** Shared buffers and registry mappings must use high-performance synchronization primitives ensuring lock contention < 5%.
4. **Resilience & Zero Silent Drops:** Transient failures are cushioned by exponential backoff; permanent failures are recorded in durable dead-letter storage with full diagnostic metadata.

---

3. Architecture: Component Model & Descriptions

3.1 Component Diagram



3.2 Detailed Component Specifications

Component 1: Ingestion API Gateway & Event Loop Controller

- **Responsibilities:** Terminates incoming TLS connections; authenticates Bearer tokens; executes schema validation; generates a unique UUIDv4 `task_id`; and enqueues the request into `service_queue`.
- **Interfaces:** Exposes HTTP REST endpoint `POST /api/v1/webhooks/ingest`.
- **Internal Mechanisms:** Operates an asynchronous event loop (epoll / kqueue / libuv); non-blocking I/O ensures ingestion p99 response times stay < 15 ms.

Component 2: In-Memory Concurrent Service Buffer (`service_queue`)

- **Responsibilities:** Serves as the central decoupling shock-absorber between high-burst ingestion and worker thread consumption.

- **Thread Safety:** Implemented as a thread-safe FIFO queue guarded by a mutex and condition variables (`std::condition_variable` / pthread conditional signals).
- **Capacity & Bounding:** Configured with a default capacity of 50,000 tasks and a maximum dynamic capacity of 100,000 tasks. When queue depth reaches the warning threshold (90% of current capacity), it logs saturation warnings and initiates upstream backpressure throttling. When queue depth reaches 100% of hard capacity, the Ingestion Gateway immediately rejects excess requests with HTTP 429 (`Retry-After: 5`).
- **Memory Optimization (`PayloadReference`):** To strictly enforce the ≤ 500 MB RSS ceiling under 50,000 tasks (where 50,000 × 64 KB inline payloads would require ~3.2 GB RAM), payloads ≤ 2 KB are retained inline, whereas payloads > 2 KB (up to 64 KB) are immediately offloaded to fast local disk/NVMe temporary staging. `service_queue` holds lightweight `WebhookTask` descriptors (~200 bytes) containing a `PayloadReference`, consuming only ~10 MB RAM for 50,000 queued items.

Component 3: Worker Thread Pool & Request Categorizer

- **Responsibilities:** Maintains an elastic pool of OS worker threads that sleep on the `service_queue` condition variable. Upon waking, a worker thread pops a task, resolves the payload via `PayloadReference`, determines whether it is `one-shot` or `recurring`, transforms the payload, computes the HMAC-SHA256 signature, and routes it accordingly.
- **Lifecycle & Elasticity:** Pre-allocates a core baseline of 4 worker threads upon daemon startup. Under sustained ingestion bursts (queue backlog > 5,000 tasks), the pool dynamically scales up to 64 active worker threads. When backlog falls below 1,000 tasks for an idle window of 30 seconds, dynamic worker threads terminate gracefully, scaling the pool back down to the baseline 4 core threads.

Component 4: Scheduler & Timer Subsystem

- **Responsibilities:** Manages the registration, tracking, and execution of recurring webhook schedules.
- **Internal State:** Maintains `scheduler_mapping`, a hash map of `schedule_id` to `TimerHandle` objects protected by a read-write lock.
- **Timer Execution:** Integrates with the host OS monotonic clock (`CLOCK_MONOTONIC`) or timer-wheel algorithm. When a timer fires, it executes the registered callback, formats the recurring payload, updates the timer handle, and deposits the payload into `dispatch_queue`.
- **Persistence & Recovery:** All recurring schedule registrations are mirrored in the persistent `recurring_schedules` SQL database. Upon service or host restart, active recurring schedules are automatically queried, reconstructed, and re-armed.

Component 5: Dispatch Buffer (`dispatch_queue`)

- **Responsibilities:** Holds fully computed, signed, and ready-to-transmit webhook payloads awaiting network transmission.
- **Thread Safety:** Thread-safe concurrent queue decoupling worker execution from outbound network I/O.

Component 6: Dispatcher Thread Subsystem

- **Responsibilities:** Pops payloads from `dispatch_queue`, manages a pool of reusable HTTP/HTTPS connections with keep-alive, resolves target URLs, and transmits payloads.
- **Timeout & Error Detection:** Implements a strict 5,000 ms socket connection and read timeout. Inspects HTTP response status codes: 2xx triggers clean closure and delivery logging; non-retryable 4xx client errors (400, 401, 403, 404, 405, 422) route directly to DLQ; retryable failures (429, 500, 502, 503, 504, connection timeouts, and connection refusals) trigger the exponential backoff retry subsystem.

Component 7: Exponential Backoff & Retry Engine

- **Responsibilities:** Implements automated retry loops for transient endpoint or network failures.
- **State Machine:** Tracks `retries` (initialized to 0) and `wait_time` (initialized to 1 second). On each transient failure (429, 5xx, timeout), it doubles `wait_time = wait_time * 2` and checks `retries < MAX_RETRIES` (default `MAX_RETRIES = 5`, yielding 1 initial attempt + 5 retries = 6 total attempts). If retries are exhausted, it transfers the task to Component 8.

Component 8: Persistent Dead Letter Storage (DLQ) & Audit Logger

- **Responsibilities:** Provides 100% durable persistence for exhausted or malformed webhooks.
- **Storage Subsystem:** Writes failure records into a relational or embedded database (SQLite/PostgreSQL) and logs structured JSON entries to an append-only audit file on disk.

- **Sanitization:** Sanitizes sensitive credentials and secrets prior to persistence using JSON-aware recursive key inspection.

Component 9: Management & Status Query API Controller

- **Responsibilities:** Exposes administrative interfaces for health inspection, queue depth telemetry, dead-letter browsing, and manual redrive dispatching.

---

4. Architecture: Architectural Patterns

4.1 Event-Driven Architecture (EDA) & Reactive Buffering

The architecture utilizes an Event-Driven reactive buffering pattern at the ingestion layer. Upstream clients emit discrete state-change events. The ingestion engine acts as an event receiver that emits events onto the internal event bus (`service_queue`). This isolates clients from the downstream delivery lifecycle and protects downstream receivers from being overwhelmed by upstream traffic surges.

4.2 Multi-Queue Producer-Consumer Pattern

The pipeline is structured as a two-stage Producer-Consumer pattern:

1. **Stage 1:** Ingestion Gateway (Producer) → `service_queue` → Worker Threads (Consumers).
2. **Stage 2:** Worker Threads / Scheduler (Producers) → `dispatch_queue` → Dispatcher Threads (Consumers).

This two-stage separation ensures that heavy payload cryptographic computation (Stage 1) is decoupled from network socket latency and HTTP connection establishment (Stage 2).

4.3 Worker Thread Pool & Shared Concurrent Task Queue Pattern

To maximize multi-core CPU utilization while preventing thread thrashing, a shared concurrent task queue with an elastic thread pool is maintained. A core baseline of 4 worker threads is pre-allocated at startup. Idle threads wait efficiently on OS condition variables with zero CPU burn. Under burst conditions exceeding 5,000 queued tasks, the thread pool dynamically spawns up to 64 active worker threads, and automatically reclaims idle worker threads back down to 4 when queue backlog subsides below 1,000 tasks for 30 seconds. Work is popped concurrently with lock contention verified below 5% under ThreadSanitizer.

4.4 Exponential Backoff & Circuit Protection Pattern

Downstream webhook endpoints frequently experience transient network partitions, server restarts, or momentary database locks. A naive retry loop firing immediately at high frequency exacerbates server outages (the "thundering herd" problem). The system implements an **Exponential Backoff pattern**:

```
T_wait = 2^tries   for tries in [0, 1, 2, 3, 4]
```

This exponentially widens the delay between retries (1s, 2s, 4s, 8s, 16s), allowing downstream services sufficient recovery windows.

4.5 Dead Letter Channel & Persistent Audit Trail Pattern

Following the canonical *Enterprise Integration Patterns* Dead Letter Channel specification, messages that cannot be delivered after maximum retry exhaustion are segregated from active queues into a durable Dead Letter Queue (DLQ). This preserves delivery guarantees, prevents poison pills from blocking active buffers, and provides administrators with an actionable audit trail for diagnosis and replay.

---

5. Architecture: Traceability to Requirements

Architectural Component / Pattern	Addressed SRS Requirements	Addressed Jira Epics & User Stories	Design Rationale & Verification
Ingestion Gateway & Event Loop	FR-01, NFR-01, NFR-02, SEC-REQ-02	EPIC-1, WACDP-4, WACDP-9	Asynchronous non-blocking architecture guarantees p99 < 15 ms at 5,000 req/s. Verified via TC-FR-01, TC-NFR-01.
Concurrent service_queue	FR-02, NFR-09, NFR-07	EPIC-1, WACDP-4, WACDP-10	Bounded mutex-guarded queue provides thread-safe buffering with < 5% lock contention. Verified via TC-FR-02, TC-NFR-09.
Worker Thread Pool & Categorizer	FR-03, FR-04, NFR-08, SEC-REQ-01	EPIC-2, WACDP-5, WACDP-10	Separates one-shot and recurring flows; signs payloads with HMAC-SHA256. Verified via TC-FR-03, TC-SEC-01.
Scheduler & Timer Subsystem	FR-05, FR-06, NFR-03	EPIC-2, WACDP-6	Thread-safe timer registry and precision callback arming ensure scheduling precision ± 10 ms. Verified via TC-FR-05.
Dispatcher Thread Subsystem	FR-07, NFR-05, SEC-REQ-03	EPIC-2, WACDP-7	HTTP keep-alive connection pooling maximizes throughput and enforces TLS 1.3 encryption. Verified via TC-FR-07, TC-SEC-03.
Exponential Backoff Engine	FR-08, NFR-04, NFR-06	EPIC-2, WACDP-7	Automates retries with mathematical delay progression (2^tries). Verified via TC-FR-08.
Persistent DLQ & Audit Logger	FR-09, NFR-04, SEC-REQ-04	EPIC-3, WACDP-8, WACDP-11	Guarantees zero data loss by committing unrecoverable failures to durable storage. Verified via TC-FR-09, TC-NFR-04.
Status Query & Redrive API	FR-10, FR-11	EPIC-3, WACDP-8	Exposes administrative endpoints for transparency, status monitoring, and DLQ reprocessing. Verified via TC-FR-10.

---

6. Architecture: Security Architecture

6.1 Defense-in-Depth Multi-Layer Model

The security architecture enforces protection across four concentric defense tiers:

```
[ Tier 1: Ingestion Perimeter ]  --> Bearer Token Auth, Token-Bucket Rate Limiter (1000/min), JSON Schema Filter
[ Tier 2: Internal Concurrency ] --> Memory Buffer Bounds, Thread Isolation, Zero Raw Pointer Leaks
[ Tier 3: Egress Transmission ]  --> HMAC-SHA256 Payload Signatures, Anti-Replay Timestamps, Mandatory TLS 1.3
[ Tier 4: Storage & Audit ]      --> Credential Sanitization, Write-Ahead Append-Only Logs, Masked DLQ Records
```

6.2 Egress Payload Signing & Idempotency Header

To satisfy SEC-OBJ-01 and SEC-REQ-01, every outbound dispatch package includes a cryptographic signature generated in accordance with the **WACDP Webhook Signing Specification** (a standard HMAC-SHA256 scheme with timestamps, avoiding RFC 7515 JWS overhead):

1. Construct the signature payload:

```
SignPayload = Timestamp + "." + SerializedBody
```

2. Compute the HMAC using the pre-shared endpoint secret:

```
Signature = HMAC-SHA256(EndpointSecret, SignPayload)
```

3. Transmit the following HTTP headers with every outbound POST request:

- `X-Webhook-Signature: t=<Timestamp>,v1=<SignatureHex>`
- `X-Webhook-Timestamp: <Timestamp>`

- `X-Webhook-ID: <task_id>`

Idempotency Support: Outbound webhook requests include a stable unique header `X-Webhook-ID: <task_id>` which is retained identically across all retry attempts. Downstream target endpoints MAY use `X-Webhook-ID` to achieve idempotent request processing and prevent duplicate side-effects. Target endpoints also verify signature authenticity and enforce a 300-second freshness window to reject replay attacks.

6.3 Ingestion Perimeter Defense & Rate Throttling

- **Authentication:** All client requests to `/api/v1/webhooks/ingest` must present an HTTP `Authorization: Bearer <token>` header. Tokens are verified using in-memory cryptographic hashes in < 2 ms.
- **Rate Throttling:** A Token-Bucket rate limiter enforces a quota of 1,000 requests per minute per API key. Excess requests are rejected immediately with HTTP 429 Too Many Requests, preventing queue exhaustion attacks.

6.4 Transport & Data-at-Rest Protection

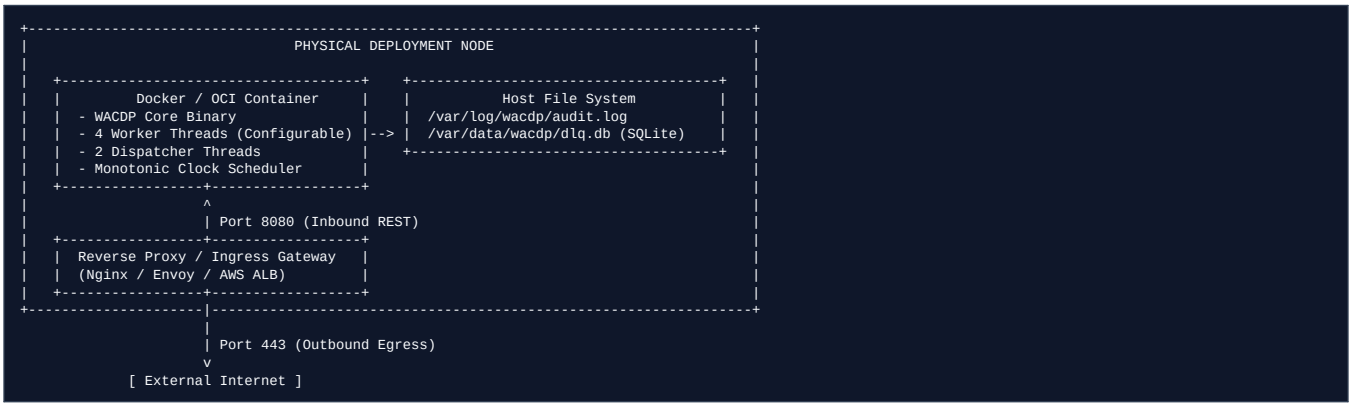
- **Transport Security (TLS 1.3 Only):** Outbound dispatches and inbound connections enforce **TLS 1.3 only** (RFC 8446) with strict SNI hostname verification (`SEC-REQ-03`). Connections utilizing TLS 1.2 or earlier and unencrypted plaintext HTTP are strictly rejected.
- **Data-at-Rest Sanitization (JSON-Aware Recursive Redaction):** Before any failed task is committed to the persistent DLQ database or audit log, a JSON-aware recursive parser walks nested payload objects and arrays, identifying sensitive keys (`password`, `secret`, `token`, `api_key`, `authorization`, `access_token`, etc.) and replaces their values with `***REDACTED***` (`SEC-REQ-04`). For non-JSON raw payloads, a secondary regex sanitizer masks query-string style credentials.

---

7. Architecture: Other Architectural Sections

7.1 Deployment & Physical Topology View

The Webhook Aggregator System is implemented exclusively in **C++20** (utilizing POSIX threads, standard library concurrency primitives, epoll/kqueue for non-blocking event loops, and OpenSSL for TLS 1.3 / HMAC-SHA256). It is packaged as a lightweight, containerized native binary deployable across modern Linux and POSIX environments:



7.2 Concurrency & Thread Synchronization Architecture

The system enforces strict multi-threading hygiene (`WACDP-10`):

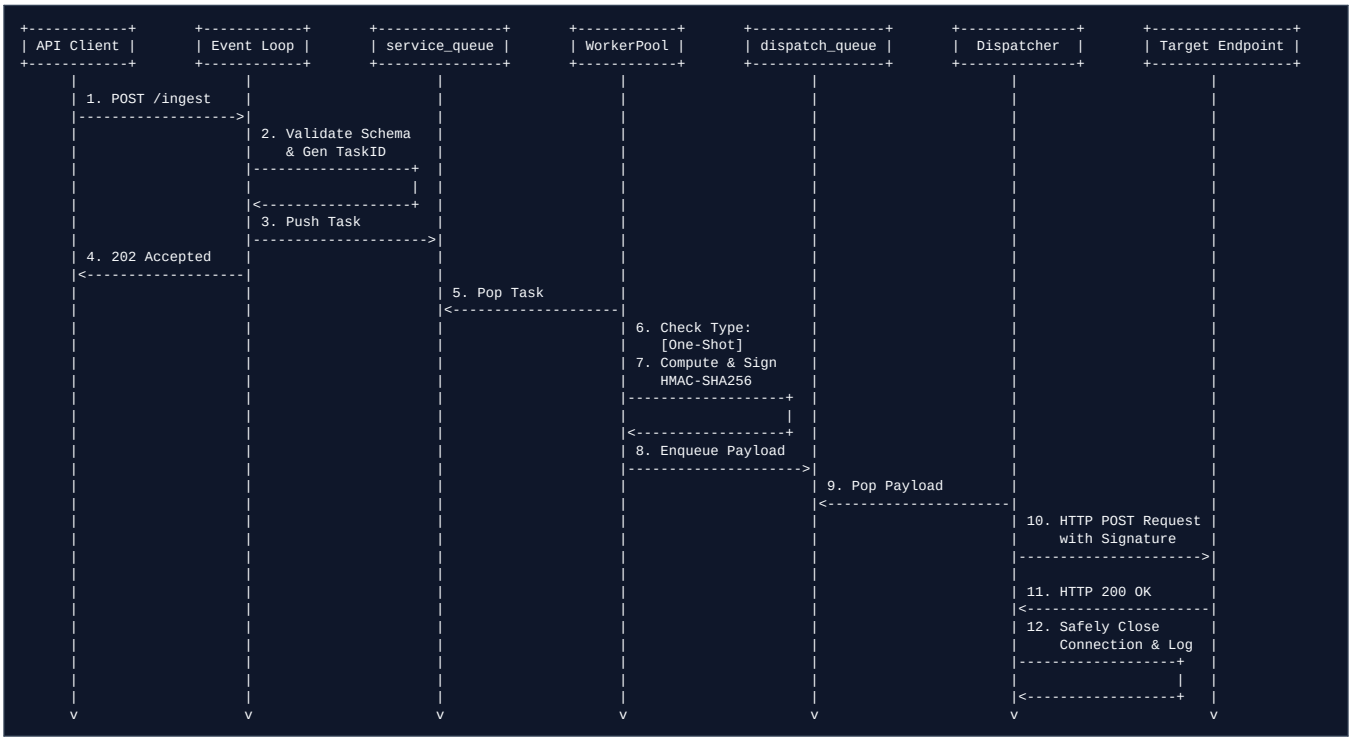
1. `service_queue` **Synchronization:** Guarded by `std::mutex` and `std::condition_variable`. Ingestion threads acquire the lock briefly (≤ 50 µs) to push task pointers and invoke `notify_one()`.
2. `dispatch_queue` **Synchronization:** Implemented as a lock-free or lightweight mutex-guarded circular ring buffer, minimizing latency between workers and network dispatchers.
3. **Scheduler Mapping Table:** Guarded by a shared reader-writer lock (`std::shared_mutex`). Read locks are acquired during timer callback lookups; write locks are acquired only during new schedule creation or deletion.

8. Design: UML Sequence Diagrams

In compliance with IEEE Std 1016 design description standards, the following two comprehensive UML sequence diagrams specify the precise runtime execution flows:

8.1 Sequence Diagram 1: One-Shot Webhook Ingestion, Buffering, Worker Processing & Dispatch

This sequence diagram illustrates the complete, normal end-to-end lifecycle of a one-shot webhook request:



8.2 Sequence Diagram 2: Scheduled Recurring Webhook, Target Outage, Exponential Backoff & DLQ Persistence

This sequence diagram depicts recurring schedule execution, subsequent target endpoint failure, the full exponential backoff retry loop, and terminal persistence into the Dead Letter Queue:



9. Design: Detailed Workflow & Activity Diagram

9.1 Multi-Swimlane Activity Diagram Analysis

The runtime workflow maps directly to the design activity diagram specified in project artifacts:

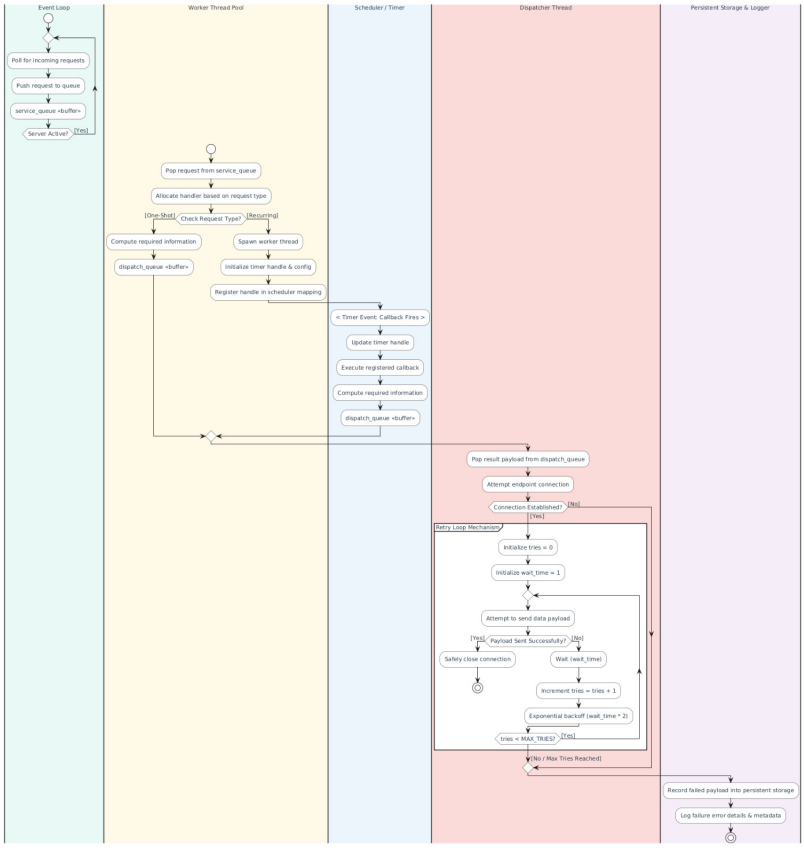


Figure: Webhook Aggregator Workflow Diagram

The system decomposes operations across **5 synchronized swimlanes**:

1. Event Loop Swimlane:

• Polls for incoming HTTP socket events using non-blocking I/O.

• Parses payload and pushes task into `service_queue <buffer>`.

• Checks if server remains active; loops continuously.

2. Worker Thread Pool Swimlane:

• Pops task from `service_queue`.

• Allocates handler based on request type.

• Evaluates branch condition:

• `[One-Shot]`: Computes required information → pushes to `dispatch_queue <buffer>`.

• `[Recurring]`: Spawns/allocates worker → initializes timer handle & configuration → registers handle in `scheduler_mapping`.

3. Scheduler / Timer Swimlane:

• Awaits timer event callback firing.

• Updates timer handle state.

• Executes registered callback → computes dynamic payload → pushes to `dispatch_queue <buffer>`.

4. Dispatcher Thread Swimlane:

• Pops result payload from `dispatch_queue`.

• Attempts endpoint TCP connection.

• If connection is established:

• Transmits initial delivery attempt (Attempt 0).

• If HTTP 2xx: safely closes connection and terminates with success.

- If HTTP 400, 401, 403, or 404 (non-retryable client error): immediately halts transmission and branches directly to persistent failure logger (DLQ).
- If transient failure (HTTP 429, 5xx, or network socket timeout): enters **Retry Loop Mechanism**:
- Initializes `retries = 0`, `wait_time = 1s`.
- Enters loop while `retries < MAX_RETRIES` (where `MAX_RETRIES = 5`, yielding 1 initial attempt + 5 retries = 6 total attempts):
- Waits `wait_time` seconds (calculated as `2^retries`).
- Increments `retries = retries + 1`.
- Doubles `wait_time = wait_time * 2`.
- Retries sending data payload.
- If successful (HTTP 2xx): exits loop, closes connection, and terminates.
- If all retries exhausted (`retries == MAX_RETRIES`): branches to persistent failure logger.
- If connection cannot be established or times out: enters retry loop; if retries exhausted, branches to persistent failure logger.

5. Persistent Storage & Logger Swimlane:

- Receives exhausted or non-retryable payload from Dispatcher Thread.
- Records failed payload into persistent durable storage (DLQ).
- Writes structured failure error details & metadata to append-only audit log → terminates.

---

10. Design: API Design Specification

The Webhook Aggregator System exposes a fully compliant RESTful API adhering to JSON specifications.

10.1 Ingestion API

- **Endpoint:** `POST /api/v1/webhooks/ingest`
- **Description:** Ingests a one-shot webhook for asynchronous delivery.
- **Request Headers:**
- `Content-Type: application/json`
- `Authorization: Bearer <API_TOKEN>`
- **Request Body:**

```
{
  "type": "one-shot",
  "target_url": "https://api.external-partner.com/v2/events",
  "payload": {
    "event_type": "invoice.generated",
    "invoice_id": "INV-2026-0901",
    "amount": 1499.00,
    "currency": "USD"
  }
}
```

- **Responses:**
- `202 Accepted`

```
 {
   "task_id": "8c42a229-8736-4d11-b0ec-698fba0e3001",
   "status": "QUEUED",
   "message": "Webhook task successfully accepted and queued for processing.",
   "created_at": "2026-09-29T11:50:00.123Z"
 }
```

- `400 Bad Request`: Schema validation failed.
- `401 Unauthorized`: Missing or invalid Bearer token.
- `429 Too Many Requests`: Rate limit exceeded (Token-bucket full).

10.2 Recurring Schedule API

- **Endpoint:** `POST /api/v1/webhooks/recurring`
- **Description:** Registers a scheduled recurring webhook execution.
- **Request Body:**

```
{
  "type": "recurring",
  "schedule": "0 */2 * * *",
  "target_url": "https://api.external-partner.com/v2/telemetry",
  "payload": {
    "probe_id": "PRB-EAST-1",
    "metric_type": "node_health"
  }
}
```

- **Responses:**
- `201 Created`

```
  {
    "schedule_id": "sch_7781a0e9",
    "status": "SCHEDULED",
    "cron_expression": "0 */2 * * *",
    "next_execution_at": "2026-09-29T12:00:00.000Z"
  }
```

- `422 Unprocessable Entity`: Invalid cron expression or interval below 1,000 ms.

10.3 Status Query API

- **Endpoint:** `GET /api/v1/webhooks/{task_id}/status`
- **Description:** Queries the current lifecycle state of a specific webhook task.
- **Responses:**
- `200 OK`

```
  {
    "task_id": "8c42a229-8736-4d11-b0ec-698fba0e3001",
    "status": "DELIVERED",
    "attempts": 1,
    "target_url": "https://api.external-partner.com/v2/events",
    "created_at": "2026-09-29T11:50:00.123Z",
    "delivered_at": "2026-09-29T11:50:00.412Z",
    "response_code": 200
  }
```

- `404 Not Found`: Task ID does not exist.

10.4 DLQ Inspection API

- **Endpoint:** `GET /api/v1/webhooks/failed?limit=50&offset=0`
- **Description:** Retrieves paginated dead-letter records for administrator inspection.
- **Responses:**
- `200 OK`

```
{
  "total_failed": 1,
  "limit": 50,
  "offset": 0,
  "records": [
    {
      "task_id": "f512c114-1928-4aa3-9218-bb10e3049182",
      "target_url": "https://flaky-endpoint.net/webhook",
      "attempts": 5,
      "last_error": "HTTP 503: Service Unavailable",
      "payload": {
        "event_type": "user.signup",
        "api_key": "***REDACTED***"
      },
      "failed_at": "2026-09-29T11:52:15.892Z"
    }
  ]
}
```

10.5 Manual Redrive API

- **Endpoint:** `POST /api/v1/webhooks/{task_id}/retry`
- **Description:** Re-enqueues an exhausted DLQ record back into the active `service_queue`.
- **Responses:**
- `200 OK`

```
{
  "task_id": "f512c114-1928-4aa3-9218-bb10e3049182",
  "status": "QUEUED",
  "message": "Task successfully extracted from DLQ and placed in service_queue."
}
```

10.6 System Health API

- **Endpoint:** `GET /api/v1/health`
- **Description:** Exposes real-time system metrics, queue depths, and thread states.
- **Responses:**
- `200 OK`

```
{
  "status": "HEALTHY",
  "uptime_seconds": 86400,
  "queues": {
    "service_queue_depth": 14,
    "dispatch_queue_depth": 3,
    "max_buffer_capacity": 50000
  },
  "threads": {
    "worker_pool_active": 4,
    "dispatcher_threads": 2
  },
  "metrics": {
    "total_ingested": 1254300,
    "total_delivered": 1254180,
    "total_dead_lettered": 120
  }
}
```

11. Design: Error Handling & Fault Tolerance

11.1 Error Classification & Fault Taxonomy

FAULT TAXONOMY		
Category	**Fault Examples**	**Architectural Handling Action**
Ingestion Faults	Malformed JSON, Payload > 64 KB	Return 400/413 immediately; drop.
	Invalid Bearer Token, Expired JWT	Return 401 immediately; drop.
	Token Bucket Limit Exceeded	Return 429 with Retry-After.
Buffer Faults	Queue Depth at 90% Warning Level	Log alert; initiate backpressure.
	Queue Depth at 100% Hard Ceiling	Return 429 (Retry-After: 5); drop.
Dispatch Faults (Transient)	TCP Refused (ECONNREFUSED)	Enter Exponential Backoff Loop.
	Read / Connect Timeout (ETIMEDOUT)	Enter Exponential Backoff Loop.
	HTTP 429 (Peer Throttling)	Enter Exponential Backoff Loop.
	HTTP 500, 502, 503, 504 Responses	Enter Exponential Backoff Loop.
Dispatch Faults (Permanent)	HTTP 400, 401, 403, 404, 405, 422	Halt retries; route to DLQ.
	DNS Unresolvable (ENOTFOUND)	Route directly to DLQ.
	Retries Exceeded (retries >= 5)	Commit to DLQ & write append log.

11.2 Exponential Backoff Algorithm Formulation

The retry subsystem executes a strictly deterministic exponential backoff algorithm:

$$T_{wait}(n) = T_{base} \times 2^n$$

Where:

- T_{base} = 1.0 second
- n = retry index in {0, 1, 2, 3, 4}
- $MAX_RETRIES$ = 5 (1 initial attempt + 5 retries = 6 total attempts)

To prevent downstream synchronization spikes (thundering herd), an absolute bounded jitter window of $J \leq \pm 100\ ms$ is applied:

$$T_{actual}(n) = T_{wait}(n) \pm UniformRandom(0, 100\ ms)$$

Where $T_{wait}(n) = 2^n$ seconds. Absolute jitter is strictly capped at ± 100 ms across all retry intervals (1s, 2s, 4s, 8s, 16s), ensuring precise backoff sequencing.

```
Attempt 0 (Initial Dispatch) : Immediate (T ≈ 0s)
Attempt 1 (Retry 1)          : Wait 1s ± 100 ms  (T_wait = 2^0 = 1s)
Attempt 2 (Retry 2)          : Wait 2s ± 100 ms  (T_wait = 2^1 = 2s)
Attempt 3 (Retry 3)          : Wait 4s ± 100 ms  (T_wait = 2^2 = 4s)
Attempt 4 (Retry 4)          : Wait 8s ± 100 ms  (T_wait = 2^3 = 8s)
Attempt 5 (Retry 5)          : Wait 16s ± 100 ms (T_wait = 2^4 = 16s)
Terminal Exhaustion          : Total 6 attempts executed; route to Persistent DLQ & Append-Only Log
```

11.3 Dead Letter Storage & Poison Pill Isolation

When a payload repeatedly causes network crashes or cannot be delivered after 1 initial attempt + 5 retries = 6 total attempts, it is classified as a "poison pill." Leaving it in in-memory queues would stall worker threads. The Dispatcher isolates it by removing it completely from `dispatch_queue` and committing the full state transactionally into the Dead Letter Queue database and appending a record to the append-only audit log.

---

12. Design: Data Dictionary & Memory Schemas

12.1 In-Memory Buffer Task Schema & Memory Bounding

To guarantee that the process RSS remains strictly ≤ 500 MB when buffering up to 50,000 tasks with payloads up to 64 KB (which would otherwise require ~3.2 GB RAM if held directly in memory), payloads ≤ 2 KB are stored inline in `inline_data`, whereas payloads > 2 KB (up to 64 KB) are immediately offloaded to disk-backed temporary storage. `service_queue` holds lightweight `WebhookTask` descriptors of ~200 bytes (~10 MB RAM for 50,000 tasks).

```
struct PayloadReference {
    bool is_inline;               // true if payload_len <= 2048 bytes
    char inline_data[2048];       // Inline buffer for small payloads (<= 2 KB)
    char storage_path[256];       // Filesystem staging path for offloaded payloads (> 2 KB)
    uint64_t file_offset;         // Byte offset in staging store
    size_t payload_len;           // Total payload length in bytes (up to 64 KB)
};

struct WebhookTask {
    char task_id[37];             // UUIDv4 string (36 chars + null terminator)
    uint8_t task_type;            // 0 = ONE_SHOT, 1 = RECURRING
    char target_url[512];         // Destination endpoint URL
    PayloadReference payload_ref; // Staged payload reference (maintains RSS <= 500 MB at 50,000 tasks)
    int64_t created_at_ms;        // Monotonic millisecond timestamp
    uint8_t retry_count;          // Current retry attempt (0..5)
    uint32_t wait_seconds;        // Current backoff sleep window
};
```

12.2 Scheduler Timer Handle Schema

```
struct SchedulerTimerHandle {
    char schedule_id[32];         // Unique identifier for schedule
    char cron_expression[64];     // Standard cron format (e.g. "0 */2 * * *")
    uint64_t interval_ms;         // Recurring interval in milliseconds
    uint64_t next_trigger_epoch;  // Next scheduled firing timestamp
    void (*callback)(void*);      // Pointer to execution callback function
    void* callback_context;       // Context pointer containing task metadata
    bool is_active;               // Boolean arming flag
};
```

12.3 Persistent DLQ Database Schema

```
CREATE TABLE dead_letter_webhooks (
    task_id VARCHAR(36) PRIMARY KEY,
    target_url VARCHAR(512) NOT NULL,
    payload_body TEXT NOT NULL,
    total_attempts INTEGER NOT NULL,
    last_error_code VARCHAR(64) NOT NULL,
    last_error_message TEXT,
    created_at TIMESTAMP WITH TIME ZONE NOT NULL,
    failed_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    status VARCHAR(20) DEFAULT 'DEAD_LETTER',
    redrive_count INTEGER DEFAULT 0
);

CREATE INDEX idx_dlq_failed_at ON dead_letter_webhooks(failed_at);
CREATE INDEX idx_dlq_status ON dead_letter_webhooks(status);
```

12.4 Persistent Recurring Schedules Database Schema & Startup Recovery

To guarantee that active recurring schedules survive daemon restarts without loss, schedule configurations are persisted in a relational store (`recurring_schedules`):

```
CREATE TABLE recurring_schedules (
    schedule_id VARCHAR(36) PRIMARY KEY,
    target_url VARCHAR(512) NOT NULL,
    cron_expression VARCHAR(64) NOT NULL,
    payload_body TEXT NOT NULL,
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMP WITH TIME ZONE NOT NULL,
    last_triggered_at TIMESTAMP WITH TIME ZONE,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_schedules_active ON recurring_schedules(is_active);
```

Daemon Startup Recovery Flow:

1. Upon daemon initialization or host restart recovery, the scheduler initialization routine queries:

```
   SELECT schedule_id, target_url, cron_expression, payload_body
   FROM recurring_schedules
   WHERE is_active = TRUE;
```

2. For each recovered active schedule, it re-populates the in-memory `scheduler_mapping`, recalculates the next execution epoch relative to the host monotonic clock, and re-arms the timer callback.
3. Recovery completes in < 3.0 seconds, restoring 100% of recurring schedules without requiring client re-registration.

End of Software Architecture & Design Specification (SADD / SDS)